

Acceso a datos: la capa ORM

Cómo una función lee y escribe en PostgreSQL sin escribir SQL. Una invocación es una transacción.

UNA INVOCACIÓN, DE PRINCIPIO A FIN



LAS PIEZAS

orm/<tabla>.py Un modelo por tabla

```
class Processes(Base):
    __tablename__ = "processes"
    id, user_id, bank_id, amount, stage, ... # columnas Mapped[...]

    get_by_id(id) # @classmethod · consulta
    list_by_user(user_id) # @classmethod · consulta
    create(**campos) # @classmethod · add + flush
    advance_to(stage) # transición de estado
```

Users Banks BankRates BearerTokens Forms Processes Files Signatures

core/db.py La sesión

- Lee host, usuario y clave de **SSM** en /cdts/<stage>/db al arrancar.
- Crea un **engine SQLAlchemy** con pg8000 que revisa la conexión antes de usarla.
- Al conectar ejecuta SET search_path TO <stage>; el código nunca nombra el schema.
- db_session** es un singleton: una sola sesión por contenedor Lambda.

El esquema viene de las migraciones Q 06

platform/migrations/sql/*.sql
un archivo por cambio, en orden de fecha

↓ al desplegar

cdts-<stage>-migrations-apply
aplica las pendientes en el schema del stage

↓

tabla en PostgreSQL
el modelo ORM la refleja: primero migración, luego modelo

- Ningún handler escribe SQL**
Todo pasa por los métodos del modelo de su tabla.
- El modelo no hace commit**
Hace flush; commit y rollback los decide @handle_exceptions.
- El código no nombra el schema**
El search_path del stage elige dev o pro.
- Credenciales solo desde SSM**
Nunca en el código ni en variables de entorno.